Radeon-Assistant

A Privacy-First Local AI Agent on AMD Radeon GPU

Project Specification Document

Track 2 - Private AI Agent Development & Local Deployment

Team: Neoh

Competition: AMD Radeon Hackathon 2026-07

Date: July 2026

Abstract. Radeon-Assistant is a fully local AI agent system running on AMD Radeon GPUs via the ROCm software stack. It combines a hand-written Planner-Executor-Reflector agent loop, a FAISS-backed RAG knowledge base, 14 built-in tools, human-in-the-loop safety controls, and a hardware-R&D specialization layer. No external API is called during inference.

1. Application Scenarios

Radeon-Assistant targets scenarios where data privacy, offline availability, and local control are non-negotiable. By running the entire inference pipeline on AMD Radeon GPUs, the system eliminates the latency, cost, and privacy risks associated with cloud-based LLM APIs.

1.1 Local Knowledge Base Q&A

Users upload private documents (PDF, DOCX, Markdown, plain text) and ask questions against them. Documents are parsed, chunked, embedded with all-MiniLM-L6-v2, and stored in a local FAISS index. Top-K chunks are retrieved as context for the LLM. Source metadata is preserved so answers can be traced back to the originating document.

1.2 Office & Desktop Automation

The agent performs file operations, executes shell commands, runs Python code, and queries system information through a registry of 14 built-in tools. High-risk operations require explicit user approval before execution.

1.3 Multi-Step Task Planning

Complex requests are handled by the Planner-Executor-Reflector loop. The Planner emits a JSON list of steps; the Executor runs each step with HITL approval; the Reflector evaluates completion and suggests retries if needed. The loop supports up to 10 iterations.

1.4 Hardware R&D Assistance (Specialization)

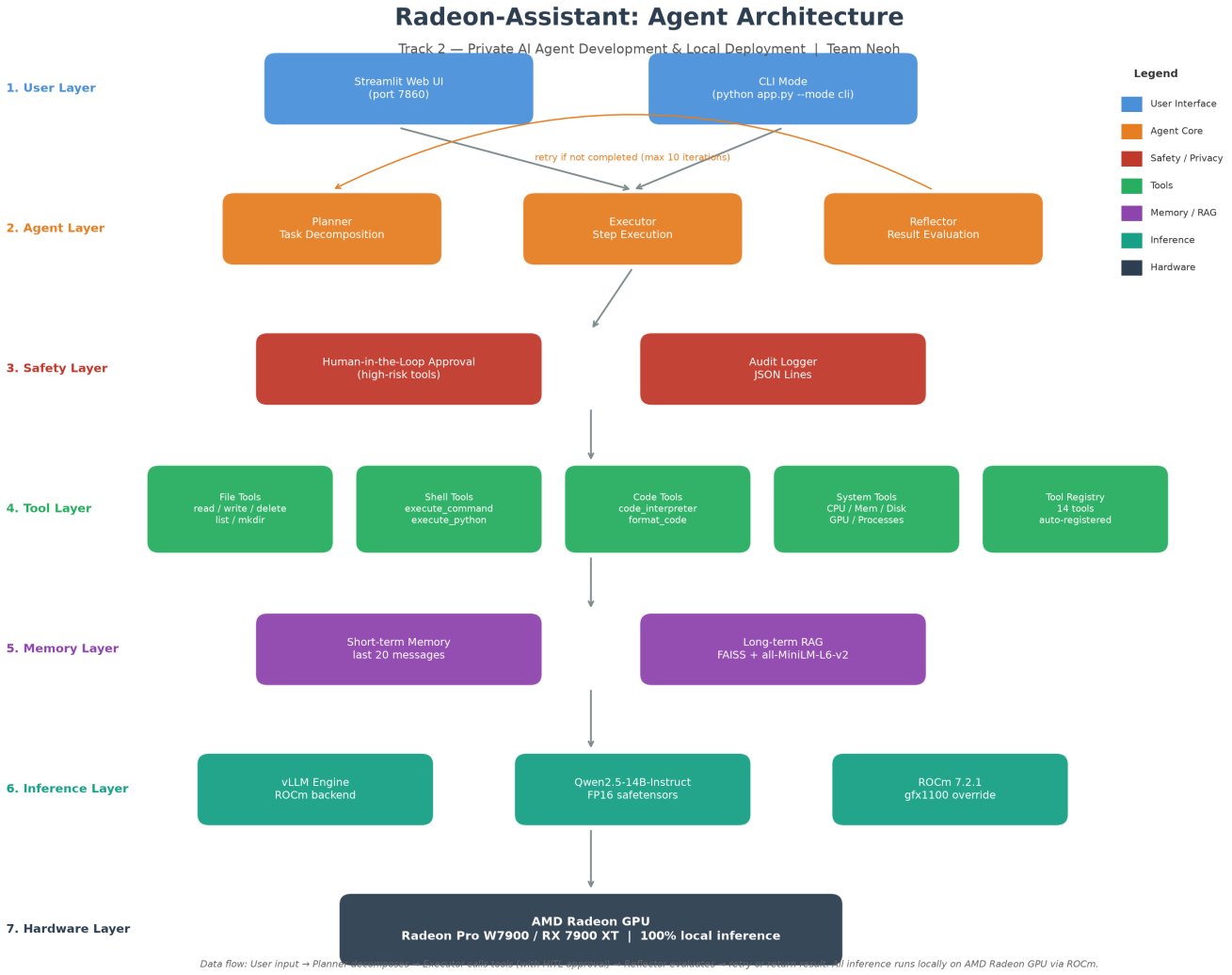
A hardware-domain system prompt and dedicated tools help engineers read chip datasheets, compare parameters, and generate Verilog / SystemVerilog modules and testbenches locally. PDF tables (pin tables, electrical-parameter tables) are extracted and indexed for RAG retrieval.

1.5 Privacy-Sensitive Environments

No component makes outbound network calls during inference. All LLM inference, embedding, and vector search run locally. An audit log records tool invocations, approval decisions, tasks, and chat metadata in JSON Lines format.

2. System Architecture

The system is organized into seven layers. Data flows from the user interface through the agent core, safety layer, tools, memory, and inference engine, ultimately reaching the AMD Radeon GPU hardware.



Layer Responsibilities

Layer	Component	Responsibility
User	Streamlit Web UI / CLI	Accepts input, displays responses, handles file uploads
Agent	Planner / Executor / Reflector	Decomposes tasks, executes steps, evaluates results
Safety	HITL Approval + Audit Log	Intercepts high-risk operations, records all actions
Tools	14 registered tools	File, shell, code, system, and hardware operations
Memory	Short-term + FAISS RAG	Conversation history and vector retrieval of documents
Inference	vLLM + ROCm	Safetensors model loading, GPU offload, chat completion
Hardware	AMD Radeon GPU	Physical compute via HIP/ROCm kernels

Agent loop detail: When a user submits a task, the Planner receives the task description and available tool schemas, then prompts the LLM to produce a JSON array of steps. The Executor processes each step, invoking the approval gate when required. The Reflector evaluates whether the task was accomplished and provides a suggestion for the next iteration if needed.

3. Core Capabilities

3.1 Local Knowledge Retrieval (RAG)

Supports PDF (pdfplumber, with table extraction), DOCX, Markdown, and plain text. Documents are split into 512-character chunks with 50-character overlap. Chunks are embedded with all-MiniLM-L6-v2 (384-dimensional vectors) and stored in FAISS IndexFlatL2. Top-5 chunks are retrieved at query time.

3.2 Tool Calling

Fourteen tools are registered at startup across five categories:

- File tools: read_file, write_file, delete_file, list_directory, create_directory
- Shell tools: execute_command, execute_python
- Code tools: code_interpreter, format_code
- System tools: get_system_info, get_gpu_info, get_process_list
- Hardware tools: generate_verilog, generate_testbench

Each tool declares its parameter schema and whether it requires approval. New tools are added by registering a single Python function.

3.3 Multi-Step Task Planning

The Planner-Executor-Reflector loop handles tasks requiring multiple tool calls. The Planner generates up to 5 steps per task; the Executor processes them with HITL approval for high-risk operations; the Reflector evaluates the outcome and guides retry if necessary.

3.4 Local Multi-Turn Memory

Short-term memory keeps the last 20 conversation messages. Long-term memory is the FAISS vector store, which persists across sessions and provides semantic retrieval of uploaded documents.

3.5 Permission & Privacy Protection

Human-in-the-loop approval intercepts tools marked requires_approval=True. The audit logger writes JSON Lines records for every tool call, approval decision, task execution, and chat event. Chat content is logged only as length summaries to protect privacy.

4. Hardware R&D Specialization

To make the 'hardware R&D private agent' claim concrete, three specialization layers were added on top of the general agent framework:

- Hardware-domain system prompt: constrains answers to EDA, MCU/SoC/FPGA, datasheets, schematics/PCB collaboration, Verilog/SystemVerilog, timing/power estimation, and embedded firmware.
- Datasheet parsing: PDF tables (pin tables, parameter tables) are extracted and indexed so they become searchable by the RAG subsystem.
- Hardware generation tools: generate_verilog and generate_testbench use the local LLM to produce synthesizable modules and testbenches, written to ./generated/.

This is a usage-scenario specialization, not a from-scratch EDA engine. Generated Verilog/testbenches are not verified by a simulator (e.g., iverilog) in the current version.

5. Model & Local Deployment Plan

5.1 Model Selection

The primary LLM is Qwen2.5-14B-Instruct in FP16 safetensors format (~28 GB). It was chosen for strong Chinese/English bilingual capability, native function-calling support, and the Apache 2.0 license. Qwen2.5-7B-Instruct is supported as a lightweight fallback (~15 GB). The embedding model is all-MiniLM-L6-v2 (23 MB, 384-dimensional vectors). The vector store is FAISS IndexFlatL2.

Component	Model / Format	Purpose
Primary LLM	Qwen2.5-14B-Instruct FP16 safetensors	Core inference, planning, tool decisions
Lightweight LLM	Qwen2.5-7B-Instruct FP16 safetensors	Faster fallback on limited VRAM
Embedding	all-MiniLM-L6-v2	Document vectorization
Vector store	FAISS IndexFlatL2	Local similarity search

5.2 Deployment Environment

Target platform: Linux (Ubuntu 22.04+) with ROCm 7.0+ and an AMD Radeon GPU. Testing was performed on AMD Radeon Cloud with a Radeon Pro W7900 (48 GB VRAM, gfx1100). vLLM is installed from the official ROCm pre-built wheel. Because W7900/RX 7900 are gfx1100 consumer/professional cards, the environment variables HSA_OVERRIDE_GFX_VERSION=11.0.0 and PYTORCH_ROCM_ARCH=gfx1100 must be set before importing vLLM/torch.

5.3 Deployment Steps

- Install dependencies: bash install_rocm.sh
- Download model: python scripts/download_model.py --model qwen2.5-14b
- (Optional) Initialize RAG: python scripts/init_rag.py or python scripts/init_hardware_rag.py
- Run CLI: python app.py --mode cli; or Web UI: python app.py --mode web --port 7860

6. Performance & Optimization

Measured on AMD Radeon Cloud (Radeon Pro W7900, 48 GB VRAM, single GPU, ROCm 7.2.1, vLLM 0.25.1):

GPU	Model	Speed	VRAM
Radeon Pro W7900	Qwen2.5-14B FP16	~27.5 tokens/s	~30 GB
Radeon Pro W7900	Qwen2.5-7B FP16	~46 tokens/s	~15 GB

Optimization notes: vLLM uses PagedAttention and continuous batching. enforce_eager=False enables CUDA/ROCm graphs. Tensor parallelism is configurable but defaults to single-GPU. For higher throughput, users can switch to 7B or explore quantization with vLLM serve.

7. Submission Structure

The submission is located in submissions/Neoh/ and includes:

- agent/ - Planner, Executor, Reflector, audit logger, prompts
- inference/ - vLLM engine wrapper and model loader
- memory/ - Document parser, FAISS vector store, memory manager
- tools/ - 14 registered tools including hardware specialization tools
- ui/ - Streamlit web interface
- scripts/ - Model downloader and RAG initialization scripts
- docs/ - architecture.png and project_spec.pdf
- config.yaml, requirements.txt, install_rocm.sh, app.py, notebooks/startup.ipynb

8. Limitations & Next Steps

The following limitations are recorded objectively to align claims with code reality:

- Verilog / testbench generation is LLM-based text generation; it is not connected to a simulator (iverilog, Verilator) for automatic verification.
- Hardware features are usage-scenario specializations on top of a general agent framework, not a from-scratch EDA engine.
- Model selection (14B default) has not yet been backed by a systematic benchmark across 7B/14B/32B.
- ROCm inference requires Linux + AMD GPU; Windows desktop cannot run the ROCm vLLM wheel directly.

Planned next steps: add a small benchmark script for 7B/14B comparison; connect generated Verilog to iverilog for smoke-test validation; and expand the hardware document corpus with real datasheets.